CHAPTER 11: THE NUMBERS BEFORE YOU BUILD

Summary: *Cost and latency questions have a characteristic wrong answer: a sensible optimisation applied to a term that was never dominant. This chapter is about finding the term that actually carries the spend or the delay before touching anything, which levers reduce cost without changing a word of what the model reads, and how the published measurements behave, including the ones that contradict the obvious intuition.*

Find the Dominant Term Before Choosing a Lever

The rule that decides more items here than any other is embarrassingly simple. **Measure which part of the workload actually carries the cost** before **selecting a way to reduce** it.

Every plausible alternative picks a lever first and never establishes that the lever touches whichever term is actually the largest for this workload. Cache the prompts. Move to a cheaper model. Shorten the output. Each is a real technique. Each is worthless if the thing eating your budget sits somewhere else entirely, and humans are remarkably confident about which one it is without looking.

Name what you are looking for: the **load-bearing control** is the one input that, changed, actually moves the number you care about. Everything else is a knob you can turn all day.

Two published findings show how badly intuition performs here.

Spend is not spread evenly. On a documented 20-problem run, 2 problems carried 43% of the total spend. The operational consequence is stated directly: compare models on the hardest tenth of your tasks rather than on a typical one, because a **failed task bills its tokens and then bills the retry**.[1]

How you **supply data can dominate everything else**. In a documented comparison over a 1,862-row public CSV, pasting the table into the prompt as roughly 91,000 input tokens answered 6 of 25 aggregate questions correctly at $5.01 per run. Uploading the file and querying it with code execution answered 25 of 25 at $0.40. Claude Opus 5 showed the same pattern at $13.45 against $1.91.[1]

The cheaper option was also the more accurate one, by a wide margin, and no quantity of prompt optimisation on the expensive path would have closed that gap. When **accuracy and cost move together**, you are looking at a **structural choice**, not a **trade-off**.

All of those figures are Anthropic-internal benchmark runs from July and August 2026 at the list prices of the time, described as directional rather than guaranteed, with the explicit instruction to measure on your own workload.[1] That caveat is worth carrying into any scenario that quotes a benchmark at you.

Two Levers Cut Cost Without Changing What the Model Reads

When a scenario carries an accuracy commitment and asks for cost reduction, the distinction that selects the answer is precise.

Some levers **change the price of a request** without **altering one byte of what the model actually reads**. Caching does this: the same content, priced differently. Batch processing does this: the same content, processed asynchronously at 50% of standard prices.[2]

Every remaining saving works by handing the model a **smaller amount of material**, a **different set of material**, or a **different model altogether**. All three put the accuracy commitment at risk, and all three therefore require evidence before they can be chosen.

That is why caching and batching keep appearing together as the pair: they are the two that cannot damage quality, because quality has no way to notice them.

The caching arithmetic is worth memorising because it produces a clean decision rule.[3]

Operation	Rate against base input	Consequence
Cache read	0.1x	Cheap enough that reads dominate the economics

Operation	Rate against base input	Consequence
5-minute cache write	1.25x	**Pays off after one read**
1-hour cache write	2x	**Pays off after two reads**

Two other pricing facts change designs and are easy to miss.

Pinning inference to the United States on **Claude 4.6 and later** applies a **1.1x multiplier** to **every pricing category**, cache writes and reads included.[3] A data-residency requirement is therefore also a 10% line item, and it is not confined to the parts you would expect.

Tools cost tokens before anyone calls them. The tool use system prompt adds 286 tokens on Claude Opus 5 for automatic or no tool choice and 406 tokens when a specific tool or any tool is required. The computer toolset adds about 4,500 input tokens to a request, and the browser toolset about 6,600.[3]

Batch processing carries three operational details beyond the 50% rate: most batches finish in under an hour, requests that reach the 24-hour expiration go unbilled, and results stay available for 29 days after the batch is created.[2]

Latency Has Two Definitions

These are defined separately and a scenario can turn on the difference.[4]

Latency is the delay between submitting a prompt and receiving the generated output.

Time to first token is how long it takes to produce the first token after the prompt is received.

Three documented approaches reduce latency, and they act on different parts of it.[5] **Choose a faster model**, with Claude Haiku 4.5 named as offering the fastest response times for speed-critical applications. **Minimise the token count** in both the

prompt and the **expected output**, including a hard cap on generated length. And **stream**, which lets the model start returning output before generation finishes.

The third one needs care, because the documentation is precise about what it improves: streaming is described as improving the **perceived** responsiveness of an application. It changes **when the human at the other end starts seeing something**. It does **not reduce how long the whole answer takes**. If a scenario measures end-to-end completion, streaming is not the answer. If it complains that the interface sits blank, it is.

The strongest latency answers cut whatever happens **ahead of the first token**, and leave the quality of the answer exactly as it stands. That framing rules out the seductive wrong answers, which buy speed by surrendering recall or capability the scenario has already told you is not needed.

Fast Mode, and the Model That Says Nothing

On Claude Opus 5 and Claude Opus 4.8, fast mode raises output tokens per second by up to 2.5x, across the whole context window, and charges premium rates for it. The model underneath is unchanged: the **same weights, a quicker inference configuration, and no difference** in what it can do.[6]

That last clause matters: fast mode is not a quality trade. It is a price trade.

Three operational details come with it.[6]

It has a dedicated rate limit, which returns a 429 carrying a retry-after header when exceeded.

Requesting it on an **unsupported model returns an error**, except on **Claude Opus 4.6**, which silently runs at standard speed while reporting the speed accurately in the response usage object. A **silent degradation with an honest usage field is a monitoring problem**: nothing errors, and the only place the truth appears is a field no human is reading.

The **SDKs retry rate-limited requests up to 2 times by default**, and there is a trap in switching that off: setting the retry count to zero also disables retries for other transient errors, including overloaded and internal server errors. You **cannot turn off just the one kind of retry**.

What the Effort Measurements Actually Show

Effort trades accuracy for cost, and the published numbers show the trade is not uniform across kinds of work.[1]

Workload	Effort setting	Documented result
Research and knowledge work	Low	Gives up 1 to 3 points for a third to a half off cost per task
Research and knowledge work	Medium	Matches the default's accuracy at 70 to 85% of its cost
Long-horizon coding, Opus 5	Medium	About 2 points given up for half the cost
Long-horizon coding, Opus 5	Low	About 8 points given up for a quarter of the cost

The shape to carry: **medium** is **close to free on knowledge work** and **expensive on long-horizon coding**. A policy that sets one effort level across an organisation is choosing to be wrong on one of those two.

Wall-clock time moves with it. On one benchmark, low effort took 4.5 minutes per problem against 7.9 at the default. On a corpus benchmark, episodes took 7.9, 9.1 and 11.4 hours at low, medium and the default.[1]

And one counter-intuitive result worth knowing, because it is the kind of thing a scenario is built from: a **re-run policy**, running cheap first and retrying failures, solved about 93% of tasks at about $0.70 each, against 91.7% at $1.39 for running everything at the default. Better and cheaper. But the documented conditions are the whole point: it requires having a **failure signal**, and it costs **two runs of wall-clock time for every first-pass failure**.[1] Without a way to detect failure, the policy is just a cheaper wrong answer.

One more measured result cuts against a common instinct. In a documented triage job, a one-line final answer used 39% fewer output tokens and cost $0.49 against the original two-line format's $0.57, while a five-section memo used six times the output tokens and cost $1.40. All three were inside run-to-run noise on accuracy.[1] The verbose format bought nothing and cost nearly three times as much.

A Refusal Is a 200, Not an Error

This behaviour is precise and produces a family of items.

A **refusal** comes back as a **successful HTTP 200 response** rather than an **error**.[7] Any handler that only inspects status codes will record a refusal as a success and pass its content downstream.

Server-side fallback retries the request on **another model** inside a **single API call**, either by default routing based on the refusal category, or against a list of up to three named models. And the trigger is narrow: **only a safety classifier decline triggers the fallback**. A rate limit, an overload or a server error on the requested model is returned as-is.[7]

There is a billing subtlety worth knowing because it explains a design that otherwise looks odd. Prompt caches are held **per model**, so retrying a refused request on a different model would normally mean writing the already-cached conversation prefix into the second model's cache at the higher write rate. A credit token minted on the refusal and echoed on the retry causes the retry to be billed as **though the conversation had been on the new model all along**.[8]

Underneath all of this sits the general rule: **every response carries a field saying why generation stopped**, with distinct values for a natural finish, hitting the output token limit, hitting a custom stop sequence, calling a tool, a paused turn, a refusal, and exceeding the context window. The documentation is explicit that this field belongs in your response-handling logic and that these values must be distinguished from actual errors.[9]

How Wrong Answers Are Built on Cost and Latency Questions

- A **lever chosen before the driver is measured**. The option names a real optimisation with no evidence it touches the dominant term. The test: does the scenario say what is **actually consuming the cost or the time**?
- A **saving that changes what the model reads**. The option shortens context, swaps content, or drops to a cheaper model where an accuracy commitment exists. The test: after this change, is the **material the model reads identical**?
- **Perceived improvement offered against a measured one**. The test: does the scenario **measure completion**, or **complain about waiting**?
- A **speculative tail used to reject a measured gain**. The option declines a demonstrated improvement by worrying about behaviour nobody measured. The test: is there a **number for the concern** being raised?
- A **status-code-only handler**. The test: what does this system do when the response is a **200 carrying a refusal**?

When the scenario says	The answer is usually	The trap is usually
Cost must come down, with an accuracy commitment in place	Caching and batching, which leave the read material identical	Trimming context, swapping content, or dropping model tier
A cost lever must be chosen	Measure which term dominates first	Committing to caching, a cheaper model or shorter output on instinct
A large table or dataset is pasted into the prompt	Upload it and query with code execution	Optimising the prompt that contains the table

When the scenario says	The answer is usually	The trap is usually
Models must be compared for cost	Compare on the hardest tenth of tasks	Comparing on a typical task, which hides that failures bill twice
The interface feels unresponsive	Streaming	Streaming when the complaint is total completion time
End-to-end time must fall	A faster model, fewer tokens in and out, a hard output cap	Streaming, which changes perception rather than duration
Output tokens per second must rise on Opus 5 or 4.8	Fast mode, at premium pricing, with no capability change	Treating fast mode as a quality trade-off
Fast mode is requested on Opus 4.6	It runs silently at standard speed; the usage object reports honestly	Assuming an error would surface the misconfiguration
Retries are being disabled	Zero also disables retries for overloaded and server errors	Expecting to switch off only rate-limit retries
A measured improvement is challenged on tail behaviour	Treat the measurement as evidence against the stated threshold	Rejecting a real gain over an unmeasured worry
A refusal must be handled	It arrives as a 200; check the stop reason	Treating refusals as HTTP errors

When the scenario says	The answer is usually	The trap is usually
A request should fall back to another model	Only a safety classifier decline triggers fallback	Expecting fallback on rate limits, overloads or server errors
Data residency pins inference to the US on 4.6 or later	Budget the 1.1x multiplier across every category	Assuming it applies only to inference
Computer or browser tools are added	Budget roughly 4,500 and 6,600 input tokens per request	Counting only the tokens you wrote
Effort policy is being set organisation-wide	Set it per workload; medium is near-free on knowledge work and costly on long-horizon coding	One level everywhere
Cheap-first with retry is proposed	Sound, but only with a failure signal and tolerance for two runs of wall-clock time	Adopting it without a way to detect failure
Cost and latency must both fall, two wanted	Reuse of an unchanged prefix, and easy traffic sent to a smaller tier	Raising the output ceiling, or adding retrieval or examples, all of which add tokens

When the scenario says	The answer is usually	The trap is usually
A cost target must be met without losing quality, two wanted	Find where the tokens go first, then cache the stable part and route by measured difficulty	Shrinking the test set or switching off tracing, which cut a different bill and remove the instrument
Profiling shows most input is a fixed block sent every call	Cache the static prefix; nothing is removed from the prompt	Compressing the variable remainder, or deleting the reference material the answer depends on

The habit worth carrying: before optimising anything, find the **two requests that cost the most** and ask why. Averages hide the term that is actually paying.

What to Remember

- **Measure which part of the workload carries the cost** before choosing a lever. Every alternative commits to a technique **without evidence it touches the dominant term**. The **load-bearing control** is the **one input that actually moves the number**.
- Spend is **not evenly distributed**: **2 problems carried 43**% of a documented 20-problem run. Compare models on the **hardest tenth of tasks**, because a failed task **bills its tokens and then bills the retry**.
- **Caching and batching** change what a request costs **without altering a byte of what the model reads**. Every other saving hands over a **smaller amount of material**, **different material**, or a **different model**, and all three endanger an accuracy commitment.
- Cache reads cost **0.1x base input**; a **5-minute write** costs **1.25x** and **pays off after one read**; a **1-hour write** costs **2x** and **pays off after two**.
- Pinning inference to the US on Claude 4.6 and later applies **1.1x to every pricing category**, cache reads and writes included.

- The tool use system prompt adds **286 tokens** on Opus 5 for automatic or no tool choice, **406 when a tool is required**; the computer toolset adds about **4,500 input tokens** and the browser toolset about **6,600**.
- Batch processing runs at **50% of standard prices**, **mostly finishes inside an hour**, does **not bill requests that reach the 24-hour expiration**, and **keeps results for 29 days**.
- Pasting a large table into the prompt was **both worse and more expensive** than uploading it and using code execution: **6 of 25 correct at $5.01** against **25 of 25 at $0.40**. When accuracy and cost improve together, the choice is **structural, not a trade-off**.
- **Latency** is the delay from prompt to output; **time to first token** is a separately defined figure. **Streaming improves perceived responsiveness** and does **not shorten total time**. Real reductions come from a **faster model**, **fewer tokens in and out**, and a **hard cap on generated length**.
- On Opus 5 and Opus 4.8, fast mode raises output tokens per second by **up to 2.5x** at premium rates, running **identical weights** on a quicker inference configuration with **no difference** in what it can do. It has its **own rate limit** returning **429 with retry-after**, and on Claude Opus 4.6 it **silently runs at standard speed** while **reporting honestly in the usage object**.
- The SDKs **retry rate-limited requests twice by default**, and setting retries to zero **also disables retries for overloaded and internal server errors**.
- **Medium effort matches default accuracy at 70 to 85% of cost** on knowledge work, but costs about **2 points on long-horizon coding**, and low costs about **8 points** there for a **quarter of the price**. **One effort level across an organisation** is wrong somewhere.
- A **cheap-first, retry-on-failure** policy reached about **93% of tasks at roughly $0.70 each** against **91.7% at $1.39**, but only with a **failure signal**, and at **two runs of wall-clock time** per first-pass failure.

- A verbose output format cost **nearly three times as much** and **bought nothing measurable**: a one-line answer used **39% fewer output tokens** than a two-line one, and a five-section memo used **six times as many**, with all three **inside run-to-run noise** on accuracy.
- A refusal is returned as a **successful HTTP 200, not an error**. **Server-side fallback retries on another model within one API call**, by refusal-category routing or a list of **up to three named models**, and **only a safety classifier decline triggers** it; rate limits, overloads and server errors come back as-is.
- Every response carries a **stop reason** distinguishing a natural finish, the output token limit, a stop sequence, a tool call, a paused turn, a refusal and an exceeded context window. **Read** it, and **distinguish those values from actual errors**.
- Published benchmark figures are **Anthropic-internal runs** from July and August 2026 at the **list prices of the time**, **directional rather than guaranteed**, with an explicit instruction to **measure your own workload**.